192425037

EXP36:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

int modinv(int a, int m) {
    int x;
    for (x = 1; x < m; x++) if ((a * x) % m == 1) return x;
    return -1;
}

int main() {
    int a, b, p, q, i;
    char text[200], enc[200], dec[200];

    while (1) {
        printf("\nEnter key a and b (0 0 to quit): ");
        scanf("%d %d", &a, &b);
        if (a == 0 && b == 0) break;

        /* C = E([a,b], p) = (a*p + b) mod 26 must be one-to-one */
        if (gcd(a, 26) != 1) {
            printf("a = %d is NOT allowed (gcd(a,26) = %d): cipher is not one-to-one.\n", a,
gcd(a, 26));
            for (p = 0; p < 26; p++)
                for (q = p + 1; q < 26; q++)
                    if ((a * p + b) % 26 == (a * q + b) % 26) {
```

```
                printf("Example: E([%d,%d], %d) = E([%d,%d], %d) = %d\n", a, b, p, a, b, q,
(a * p + b) % 26);
                p = q = 26;
            }
        continue;
    }
    printf("a = %d is valid (gcd(a,26) = 1).\nEnter plaintext: ", a);
    scanf("%s", text);
    for (i = 0; text[i]; i++) enc[i] = (a * (toupper(text[i]) - 'A') + b) % 26 + 'A';
    enc[i] = '\0';
    for (i = 0; enc[i]; i++) dec[i] = (modinv(a, 26) * ((enc[i] - 'A' - b + 26) % 26)) % 26 + 'a';
    dec[i] = '\0';
    printf("Ciphertext: %s\nDecrypted : %s\n", enc, dec);
  }
  return 0;
}
```

OUTPUT:

```
Enter key a and b (0 0 to quit): 2 3
a = 2 is NOT allowed (gcd(a,26) = 2): cipher is not one-to-one.
Example: E([2,3], 0) = E([2,3], 13) = 3

Enter key a and b (0 0 to quit): 5 8
a = 5 is valid (gcd(a,26) = 1).
Enter plaintext: HELLO
Ciphertext: RCLLA
Decrypted : hello

Enter key a and b (0 0 to quit): 0 0
```